garbage collector reclaim memory and prevent leaks. Listing 3.4 is a revised version that minimizes memory retention.

Listing 3.4 Closure—with memory clearing

```
function createHandler() {
  const largeObject = new Array(1000000).fill('*');
  return function handler() {
    console.log('Handler active');
    largeObject.length = 0; #A
  };
}
const handler = createHandler();
document.addEventListener('click', handler);
document.removeEventListener('click', handler); #B
```

WARNING

The JavaScript garbage collector can only reclaim memory when there are no active references to an object. Closures, if not managed carefully, may hold those references longer than necessary.

For this topic, are you able to answer these follow-up/probing questions?

- "How does JavaScript’s garbage collector determine whether a closure variable is eligible for cleanup?"
- "What strategies can you use to manage memory when using closures with event listeners?"
- "Would using WeakMap or WeakRef help in this scenario? Why or why not?"
- "How does this issue relate to single-page applications (SPAs) with frequent DOM updates and dynamic event bindings?"
- "Can you design a utility that safely registers and unregisters event listeners with closures to avoid leaks?"

3.2 Functional programming and functionality

Functional programming (FP) is a core paradigm in modern JavaScript development, and interviewers often use it to assess a candidate’s ability to